

INTELIGENCIA ARTIFICIAL (1INF24)



UNIDAD 1: Introducción a la IA. Búsqueda y optimización en IA

**Tema 3: Algoritmos heurísticos, metaheurísticos y bioinspirados
(Parte 1)**

MSc. Luis Muroya

Adaptado de material: Dr. Edwin Villanueva

Contenido

- Algoritmos heurísticos de búsqueda y optimización local
 - Hill-climbing
 - Simulated annealing

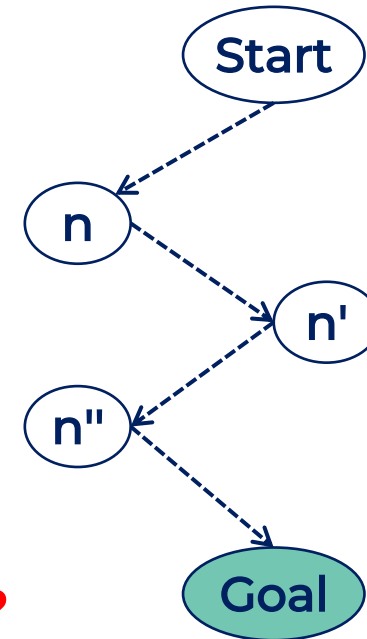
Recapitulando el Tema 2

- **Agentes de resolución de problemas:** percepción, formulación, solución (búsqueda), acciones.
- **Formulación de problema:** estados, objetivo, prueba-objetivo, acciones, transiciones, costos.
- **Búsqueda con información:** busca solución con información externa a la formulación del problema.
- **Heurística $h(n)$:** camino más barato para llegar al objetivo desde el nodo n .
- Algoritmos (implementar como *tree-search* o **graph-search*):
 - **Greedy Best-First Search:** completo*, $O(b^m)$ en tiempo y espacio. No garantiza optimalidad. $f(n) = h(n)$.
 - **A* Search:** completo en espacio finito, óptimo, exponencial en tiempo y espacio. $f(n) = g(n) + h(n)$.



¿Qué hemos visto hasta ahora?

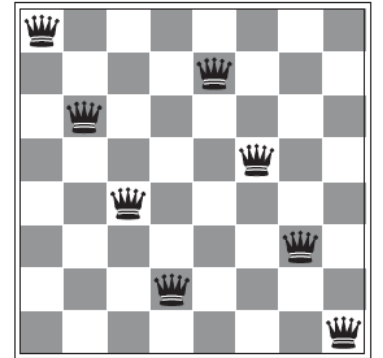
- Interés en encontrar el camino hasta el objetivo.
- Necesidad de mantener en memoria:
 - Nodos frontera
 - Nodos visitados (graph-search)
- Solución = camino hacia estado objetivo.



¿Nos interesa siempre conocer el camino hacia el objetivo?

Búsqueda local y optimización

- En muchos problemas de optimización el camino hasta el objetivo es irrelevante:
 - Cuando se quiere encontrar solo una configuración objetivo, sin importar la secuencia de acciones.
- En ese caso se puede usar algoritmos de búsqueda local:
 - **No se mantiene un árbol de búsqueda**, sólo un nodo con el estado actual.
 - Iterativamente se modifica el estado actual usando información de los estados vecinos.



Búsqueda local y optimización

Ejemplo: *Problema de las n-reinas*

- Colocar n reinas en un tablero $n \times n$, tal que cada fila, columna y diagonal tenga solo una reina:



Búsqueda local y optimización

8	4	5	9	6	7	3	1	2
7	2	3	8	1	4	9	5	6
9	6	1	5	3	2	7	4	8
3	5	9	1	7	8	6	2	4
6	1	7	2	4	3	8	9	5
2	8	4	6	5	9	1	3	7
5	7	6	4	9	1	2	8	3
4	9	2	3	8	6	5	7	1
1	3	8	7	2	5	4	6	9

Ejemplo: *Sudoku*

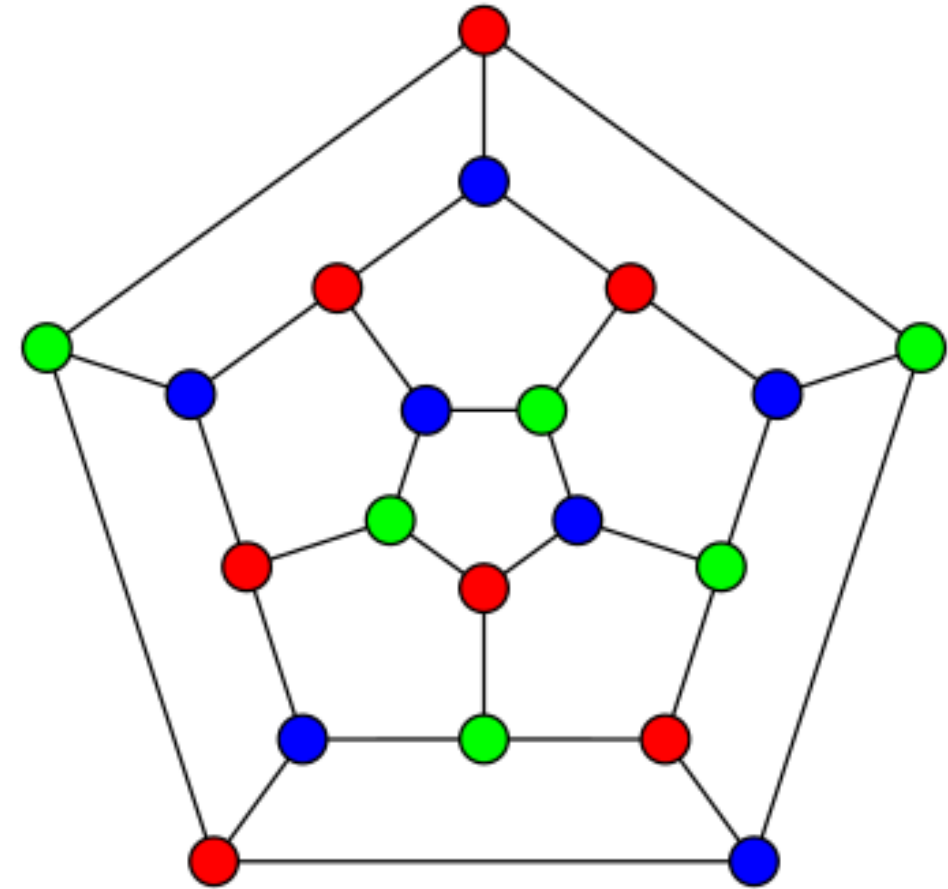
Completar una cuadrícula de 9x9 con números del 1 al 9 sin repetir en filas, columnas ni bloques.



Búsqueda local y optimización

Ejemplo: *Coloración de grafos*

Asignar colores a los vértices de un grafo tal que nodos adyacentes no compartan un mismo color.



Búsqueda local y optimización

Características de métodos de búsqueda local:

- Usan muy poca memoria, usualmente una cantidad constante
- A menudo encuentran soluciones razonables en espacios de estados enormes o infinitos (continuos), donde los métodos que exploran caminos no funcionarían.
- Pueden ser útiles para resolver problemas de **optimización** puros, donde se necesita encontrar el mejor estado de acuerdo a una **función objetivo**.



Búsqueda local y optimización

Función objetivo y función de costo

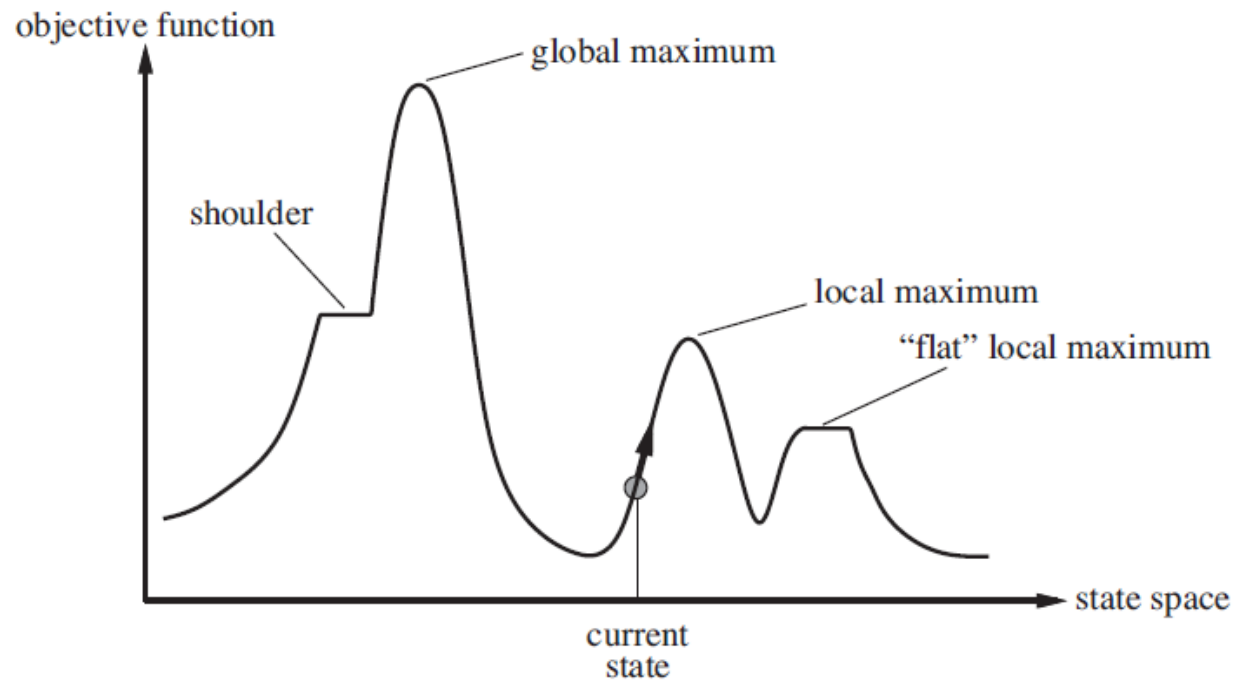
- **Función de costo:** evalúa qué tan buena es una solución. Guía el movimiento entre estados y se minimiza.
- **Función objetivo:** es lo que se desea optimizar y puede incluir a la función de costo además de otros términos. Dependiendo su naturaleza, se buscará minimizar (distancia total) o maximizar (retorno de inversión).



Búsqueda local y optimización

Superficie de espacio de estados

La superficie tiene localización (estado) y elevación (definido por la función objetivo o costo)



Búsqueda “Hill-Climbing”

(Greedy local search)

- Usa un solo nodo que mantiene el estado actual y el valor de la función objetivo o función de costo
- Solo examina los valores de los estados vecinos al estado actual
- Continuamente se mueve en la dirección donde se obtiene el mayor incremento si es función objetivo o mayor decremento si es función costo
- Termina cuando se encuentra un pico (o valle) donde ningún estado vecino tiene más alto (más bajo) valor

Búsqueda “Hill-Climbing” (Greedy local search)

Implementación (pseudocódigo):

```
function HILL-CLIMBING(problem) returns a state that is a local maximum  
  current  $\leftarrow$  MAKE-NODE(problem.INITIAL-STATE)  
  loop do  
    neighbor  $\leftarrow$  a highest-valued successor of current  
    if neighbor.VALUE  $\leq$  current.VALUE then return current.STATE  
    current  $\leftarrow$  neighbor
```

Búsqueda “Hill-Climbing”

(Greedy local search)

Limitaciones:

- Puede no encontrar la solución óptima por las siguientes razones:
 - **Máximos locales:** picos de la superficie del espacio de estados donde ningún estado vecino tiene mayor valor.
 - **Crestas (ridges):** secuencias de picos que son muy difíciles de explorar.
 - **Mesetas (plateau):** áreas planas del espacio de estados donde hill-climbing se quedaría perdido.



Búsqueda “Hill-Climbing” (Greedy local search)

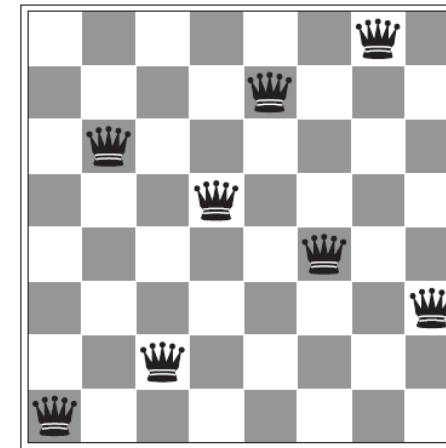
Ejemplo: Problema de las 8 reinas

- Imaginemos el problema de las 8 reinas con función de costo:
 h = número de pares de reinas que se atacan mutuamente
- Sucesores:** todos los posibles estados generados de mover una simple reina de un casillero a otro en la misma columna.

Valores de h de todos los sucesores posibles del estado actual.

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♚	13	16	13	16
♚	14	17	15	♚	14	16	16
17	♚	16	18	15	♚	15	♚
18	14	♚	15	15	14	♚	16
14	14	13	17	12	14	12	18

$h = ?$



Un posible mínimo local con $h = 1$

Existe una probabilidad de 84% de atracarse en mínimo local!



Búsqueda “Hill-Climbing” (Greedy local search)

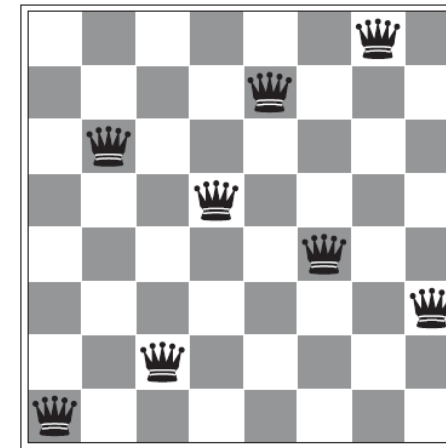
Ejemplo: Problema de las 8 reinas

- Imaginemos el problema de las 8 reinas con función de costo:
 h = número de pares de reinas que se atacan mutuamente
- Sucesores:** todos los posibles estados generados de mover una simple reina de un casillero a otro en la misma columna.

Valores de h de todos los sucesores posibles del estado actual.

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♔	13	16	13	16
♔	14	17	15	♔	14	16	16
17	♔	16	18	15	♔	15	♔
18	14	♔	15	15	14	♔	16
14	14	13	17	12	14	12	18

$h = 17$



Un posible mínimo local con $h = 1$

Existe una probabilidad de 84% de atracarse en mínimo local!



Búsqueda “Hill-Climbing”

(Greedy local search)

Posibles mejoras :

- **Movimientos laterales** para evitar mesetas
 - Sin embargo, puede ocurrir repetición infinita. Hay que imponer un límite para el número máximo de movimientos laterales.
- **Stochastic hill-climbing**
 - Escoge aleatoriamente al sucesor con probabilidad proporcional al aumento de la función objetivo.
- **Reinicios aleatorios** (*Random-restart Hill-climbing*)
 - Ejecutar varias búsquedas a partir de varios estados iniciales escogidos aleatoriamente.
 - Es completa, pues en el peor de los casos irá acabar generando el estado objetivo como estado inicial; sin embargo, es ineficiente.



Búsqueda “Simulated Annealing”

(Templado/Recocido Simulado)

- “Hill-climbing”: movimiento en sentido cuesta arriba, riesgo permanente de estancarse en un máximo local.



- “Random-walk”: movimiento aleatorio sin ver valor del vecino, puede llegar al máximo global, pero muy ineficiente.

Simulated Annealing es un punto medio entre ambas estrategias
(+completo, +eficiente)

Búsqueda “Simulated Annealing”

(Templado/Recocido Simulado)

- **Intuición:** analogía con una superficie con muchas hendiduras: alto riesgo de estancarse en mínimo local.
- **Método:** simula el proceso de templado de metales: al inicio se da una gran temperatura para luego ir bajando gradualmente:
 - Gran temperatura significa una alta probabilidad de que estados vecinos con pobre evaluación puedan ser escogidos como sucesores. Esto le permite escapar de óptimos locales. *“Agitar la superficie”*
 - Esa probabilidad disminuye con el pasar del tiempo, al disminuir la temperatura.

Búsqueda “Simulated Annealing” (Templado/Recocido Simulado)

Implementación (pseudocódigo):

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

inputs: *problem*, a problem

schedule, a mapping from time to “temperature”

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

for $t = 1$ **to** ∞ **do**

$T \leftarrow \text{schedule}(t)$ $\longrightarrow$ Control de temperatura

if $T = 0$ **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

$\Delta E \leftarrow \text{next.VALUE} - \text{current.VALUE}$

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* only with probability $e^{\Delta E/T}$ $\left. \vphantom{\begin{array}{l} \text{if } \Delta E > 0 \text{ then } \text{current} \leftarrow \text{next} \\ \text{else } \text{current} \leftarrow \text{next} \text{ only with probability } e^{\Delta E/T} \end{array}} \right\}$ Evalúa si mejora o no objetivo.

$\longrightarrow$ Actualizar nodo para probabilidad menor a B

Búsqueda “Simulated Annealing”

(Templado/Recocido Simulado)

Propiedades:

- Es posible probar que si T decrece lo suficientemente lento, la búsqueda puede encontrar una solución óptima global con probabilidad tendiendo a 1 (prop. de dist. Boltzman).
- *Simulated Annealing* es muy usada en software de diseño de circuitos integrados (ubicar componentes), optimización de redes de telecomunicaciones (ubicar antenas), *planificación* de instalaciones industriales (ubicar objetos), etc.

Bibliografía



Capitulo 4.1 del libro:

Stuart Russell & Peter Norvig “Artificial Intelligence: A modern Approach”, Prentice Hall, Third Edition, 2010

¡Gracias!

